

C++ Notes: Formatted I/O Operations

Formatted I/O means **controlling how data is read from input and how it is displayed on the output screen.**

In C++, the `<iomanip>` header provides several useful manipulators such as:

- `setw()`
 - `setprecision()`
 - `setfill()`
 - `fixed`
 - `left`
 - `right`
-

1. `setw()` — Set Width

`setw()` is used to specify the **minimum field width** for input or output.

Syntax

`setw(n)`

where `n` is the width.

Include:

```
#include <iomanip>
```

A. `setw()` with Output

```
cout << setw(10) << 42;
```

The output occupies a field of **10 characters**.

```
    42
```

There are 8 spaces before `42`.

Important Point

`setw()` specifies **minimum width**, not maximum width for output.

```
cout << setw(5) << 123456;
```

Output:

```
123456
```

The complete value is printed because it is already wider than 5.

2. `setw()` with Input

`setw()` can also control the amount of data read.

Example:

```
char name[50];
```

```
cout << "Enter your name: ";  
cin >> setw(10) >> name;
```

For a character array, `setw(10)` allows **up to 9 characters**, because one character is reserved for the terminating `'\0'`.

Example

Input:

Alexander

Output stored in `name`:

Alexande

So remember:

For a `char` array, `setw(n)` generally reads at most **`n - 1` characters**.

3. `setprecision()` — Control Decimal Precision

`setprecision()` is used to control the number of digits displayed.

```
cout << setprecision(2) << 3.14159;
```

Output:

3.1

Why?

Without `fixed`, `setprecision()` specifies the **number of significant digits**, not simply the number of digits after the decimal point.

`fixed + setprecision()`

For controlling digits **after the decimal point**, use:

```
cout << fixed << setprecision(2) << 3.14159;
```

Output:

3.14

Example

```
double salary = 45678.5678;
```

```
cout << fixed << setprecision(2) << salary;
```

Output:

45678.57

Interview Point

`setprecision(2)`

→ significant digits when used alone.

`fixed << setprecision(2)`

→ exactly **2 digits after decimal**.

4. `setfill()` — Fill Empty Spaces

`setfill()` specifies which character should be used to fill empty spaces created by `setw()`.

Example

```
cout << setw(10) << setfill('*') << 42;
```

Output:

*****42

Without `setfill()`:

```
cout << setw(10) << 42;
```

Output:

42

With `setfill('*')`:

*****42

5. left and right

By default, numbers are generally aligned to the **right**.

Right Alignment

```
cout << setw(10) << right << 42;
```

Output:

42

Left Alignment

```
cout << setw(10) << left << 42;
```

Output:

42

This is very useful when creating **tables and reports**.

6. Complete Example

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    double pi = 3.14159;

    cout << "PI: "
         << fixed
         << setprecision(2)
         << pi << endl;

    cout << setw(10)
         << setfill('*')
         << 42 << endl;

    return 0;
```

```
}
```

Output

```
PI: 3.14  
*****42
```

7. Formatted Input vs Formatted Output

Manipulator	Purpose	Example
<code>setw()</code>	Set field width	<code>setw(10)</code>
<code>setprecision()</code>	Control precision	<code>setprecision(2)</code>
<code>fixed</code>	Fixed decimal notation	<code>fixed</code>
<code>setfill()</code>	Fill empty width	<code>setfill('*')</code>
<code>left</code>	Left alignment	<code>left</code>
<code>right</code>	Right alignment	<code>right</code>

8. Important Difference: `setw()` vs `setfill()`

Think of them as a **team**:

```
setw(10)
```

says:

```
"I need a box of width 10."
```

```
setfill('*')
```

says:

```
"If the box has empty space, fill it with *."
```

Example:

```
cout << setw(10) << setfill('*') << 42;  
*****42  
└── 10 characters ──┘
```

9. `setw()` is Reset After One Operation

This is a very important interview point.

```
cout << setw(10) << 42 << 100;
```

Output:

```
42100
```

`setw(10)` applies only to the **next input/output operation**.

But manipulators such as:

```
setfill('*')  
fixed  
setprecision(2)
```

remain active until changed.

10. Formatted I/O Using a Student Example

```
#include <iostream>  
#include <iomanip>  
using namespace std;  
  
int main()  
{  
    string name = "Rahul";  
    int roll = 101;  
    double marks = 87.4567;  
  
    cout << left << setw(15) << "Name"  
        << setw(10) << "Roll"  
        << setw(10) << "Marks" << endl;  
  
    cout << left << setw(15) << name  
        << setw(10) << roll  
        << fixed << setprecision(2) << marks << endl;  
  
    return 0;  
}
```

Output

Name	Roll	Marks
Rahul	101	87.46

This is how formatted I/O can be used to create **professional-looking tables**.

★ Placement / Interview Quick Revision

Remember this simple formula:

setw() → Width
setfill() → Filling character
setprecision() → Precision
fixed → Digits after decimal
left/right → Alignment

Most common interview question

Q: What is the difference between `setw()` and `setprecision()`?

Answer:

`setw()` controls the **field width**, whereas `setprecision()` controls the **precision of floating-point output**.

And one small C++ trap worth remembering:

`setprecision(2)`

does **not necessarily mean 2 digits after the decimal**.

For that, use:

`fixed << setprecision(2)`

That tiny `fixed` can save a surprising amount of interview pain. 😊

```
#include <iostream>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    for(int i=1;i<=5;i++)
```

```
    {
```

```

        cout<<"\n"<<setw(i)<<setfill('x')<<"";

        //          1          X
        //          2          XX
        //          3          XXX
    }

    /*cout << left << setw(15) << "Name"

        << setw(10) << "Roll"

        << setw(10) << "Marks" << endl;

    cout << left << setw(15) << name

        << setw(10) << roll

        << fixed << setprecision(2) << marks << endl;
    */

    return 0;
}

#include <iostream>

#include <iomanip> // Required for setw(), setfill()

using namespace std;

int main()
{
    int n = 5;

    for(int i = 1; i <= n; i++)
    {
        string stars = "";

        // Create i stars

```



```
for(int j = 1; j <= i; j++)  
{  
    stars = stars + "*";  
}  
  
// Print the complete row right aligned  
cout << right << setw(n) << stars << endl;  
}  
  
return 0;  
}
```